

13-Tarefa: Avaliação de Afinidade

Metodologia

Nesta tarefa, testamos o impacto de diferentes afinidades de *threads* geradas pelo OpenMP no desempenho do algoritmo solver de Navier-Stokes. Utilizamos o cluster supercomputador NPAD (partição intel-128 com 1 nó computacional) processando com 16 threads via OMP.

Afinidade de *threads* diz respeito à forma como as threads de execução estão amarradas (bind) à hierarquia física do processador (Cores, Sockets, Hardware Threads).

As diretivas avaliadas foram:

- **OMP_PLACES:** cores, sockets, threads
- **OMP_PROC_BIND:** close (threads adjacentes entre si) e spread (espalhadas para distribuição equilibrada nas caches e controle térmico).

Resultados Obtidos

Abaixo apresentamos os resultados coletados com 16 threads executando a versão static no nó do NPAD:

Threads	Afinidade (Places / Bind)	Tempo (s)	Speedup (T1/Tn)
1	Padrão (Serial)	17.402	1.00x
16	Padrão	1.787	9.73x
16	cores / close	1.790	9.72x
16	cores / spread	1.784	9.75x
16	sockets / close	1.788	9.73x
16	sockets / spread	1.788	9.73x
16	threads / close	1.946	8.94x
16	threads / spread	1.785	9.75x

Discussão

Através dos resultados colhidos do cluster, podemos destacar os seguintes pontos:

- **Escalabilidade Excepcional:** Comparando com o tempo puramente sequencial (17.40s) o uso maciço das 16 threads entregou expressivos 9.7x de speedup em média.
- **close vs spread:** Na configuração mais crítica (limitando à threads), a afinidade close obteve uma severa penalidade em relação à spread (1.94s vs 1.78s). Em arquiteturas robustas baseadas em chiplets/sockets grandes do NPAD, aglomerar agressivamente workers na mesma região física da arquitetura gera gargalos nas

caches compartilhadas unificadas, superlotando os barramentos de hardware e as ULA (Unidades de Lógica). Em contraste, o uso espalhado (spread) balanceou os estêncils e aliviou a carga L2/L3.

- **libgomp e OMP_PLACES:** Observamos na saída (libgomp: Invalid value for environment variable OMP_PLACES com valor nulo) que a ausência de parametrização clara na variável devolve restrições de parser da versão utilizada pelo compilador GNU do cluster, embora o tempo padrão do Linux tenha operado eficazmente. As definições nominais cores e sockets provaram-se consistentes e equivalentes em suas margens estritas de latência na partição testada, oscilando entre os 1.78~1.79s.

Script de Submissão (job.sh)

```
#!/bin/bash
#SBATCH --job-name=navier_afinidade
#SBATCH --time=0-1:00
#SBATCH --partition=intel-128
#SBATCH --nodes=1
#SBATCH --cpus-per-task=16
#SBATCH --hint=compute_bound

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
make clean
make

echo "====="
echo "Avaliação de Afinidade (Task 13)"
echo "Número máximo de threads do Job: $SLURM_CPUS_PER_TASK"
echo "====="

EXEC="./navier_static"
PLACES=("cores" "sockets" "threads")
BINDS=("close" "spread")

for THREADS in 4 8 16; do
    export OMP_NUM_THREADS=$THREADS
    echo -e "\n====="
    echo "Testando com $THREADS Threads"
    echo "====="

    export OMP_PROC_BIND=false
    export OMP_PLACES=
    echo "--- Afinidade PADRÃO (Sem binding explícito) ---"
    $EXEC $THREADS

    for PLACE in "${PLACES[@]}; do
        for BIND in "${BINDS[@]}; do
            export OMP_PLACES=$PLACE
            export OMP_PROC_BIND=$BIND
```

```
        echo "--- Afinidade: PLACES=$PLACE, BIND=$BIND ---"
        $EXEC $THREADS
    done
done
done
```